When building an application for someone navigating an oppressive or controlling relationship, **safety and privacy architecture must come first.**
If an app like this is discovered by a controlling partner, it can escalate risk. Therefore, absolute discretion, ironclad local encryption, and emergency escape features aren't just "nice-to-haves"—they are vital safety prerequisites.
Here is the recommended **Privacy & Disguise Architecture** for Android, complete with code implementation.

🔒 Core Safety Architecture

1. The Decoy Vault (Disguise Mode)

- **The Concept:** The app icon and name appear as something entirely innocent on the home screen—such as a standard Notepad, Recipe Book, or Calculator.
- **The Dual-Pin/Biometric Entry:** Opening the app brings up the decoy app. Only entering a specific hidden PIN code, or using a designated biometric fingerprint, unlocks the real "Mute Point" secure interface.

2. Panic Button & Quick Exit

- **The Concept:** A fast gesture (like triple-tapping the top corner or a hardware volume button sequence) instantly closes the app, wipes recent activity from the Android Recents tray, and switches back to the decoy screen.

3. Zero-Cloud Local Encryption

- **The Concept:** No audio journals, logs, or transcripts ever touch a remote server or cloud backup. Everything is stored locally on the device using **AES-256 encryption** via SQLCipher and Jetpack Security.

4. Discreet Crisis Support Access

- **The Concept:** Quick, one-tap access to confidential support lines (such as **1800RESPECT** in Australia) integrated discreetly within the settings menu, alongside an option to instantly clear call logs.

💻 Implementation: Vault & Decoy Navigation Scaffold

Here is how to set up a secure entry gate in Jetpack Compose that toggles between a Decoy screen (e.g., a simple notepad) and the secure app based on authentication state.

presentation/vault/VaultContainer.kt

```Kotlin
package com.mutepoint.app.presentation.vault

import androidx.compose.animation.Crossfade
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
```

```
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.unit.dp

@Composable
fun VaultContainer(
    onUnlockAuthenticated: @Composable () -> Unit
) {
    var isUnlocked by remember { mutableStateOf(false) }
    var pinInput by remember { mutableStateOf("") }
    var showError by remember { mutableStateOf(false) }

    // Secret master PIN (in production, store securely via EncryptedSharedPreferences)
    val secretPin = "7412"

    Crossfade(targetState = isUnlocked, label = "VaultTransition") { unlocked ->
        if (unlocked) {
            // Load the actual sensitive application flow
            onUnlockAuthenticated()
        } else {
            // Decoy Interface: Looks like a simple, harmless Notepad app
            Scaffold(
                topBar = {
                    TopAppBar(
                        title = { Text("My Notes & Ideas") }
                    )
                }
            ) { padding ->
                Column(
                    modifier = Modifier
                        .fillMaxSize()
                        .padding(padding)
                        .padding(24.dp),
                    horizontalAlignment = Alignment.CenterHorizontally,
                    verticalArrangement = Arrangement.Center
                ) {
                    OutlinedTextField(
                        value = pinInput,
                        onValueChange = {
                            pinInput = it
                            showError = false
```

```
                    if (it == secretPin) {
                        isUnlocked = true
                        pinInput = ""
                    }
                }
                label = { Text("Quick Note / Search") }
                visualTransformation = PasswordVisualTransformation()
                singleLine = true
                modifier = Modifier.fillMaxWidth()
            )

            if (showError) {
                Spacer(modifier = Modifier.height(8.dp))
                Text(
                    text = "Note not found."
                    color = MaterialTheme.colorScheme.error
                    style = MaterialTheme.typography.bodySmall
                )
            }

            Spacer(modifier = Modifier.height(24.dp))

            // Harmless decoy placeholder content
            Text(
                "No saved notes found. Type above to create a new note."
                style = MaterialTheme.typography.bodyMedium
                color = MaterialTheme.colorScheme.outline
            )
        }
    }
}
```

🛠️ Encrypting Local Storage (Room + SQLCipher)

To ensure that any recorded audio logs or reality-check journals cannot be read if someone accesses the device file system, configure Room with an encrypted database driver:

1. Add the SQLCipher dependency in build.gradle.kts:

   Kotlin

   ```
   implementation("net.zetetic:android-database-sqlcipher:4.5.4")
   implementation("androidx.sqlite:sqlite:2.4.0")
   ```

2. Generate a secure passphrase using MasterKeys from Jetpack Security and pass it to the Room database builder:

```
Kotlin
val passphrase =
SQLiteDatabase.getBytes("YOUR_SECURE_DERIVED_PIN".toCharArray())
val factory = SupportOpenHelperFactory(passphrase)

Room.databaseBuilder(context, MutePointDatabase::class.java, "mute_point_secure.db")
    .openHelperFactory(factory)
    .build()
```

Would you like to explore how to implement the **Reality-Anchor Journaling** module next, keeping this secure vault architecture in mind?